Web Engineering & Development (SWE 363)

MongoDB

In today's lecture:

- MongoDB
- Mongoose

Reference:

- Zybook: 7.1, 7.2

MongoDB

What is MongoDB?

MongoDB is a **NoSQL document database**

- Stores data as **documents** (JSON-like format)
- **Flexible schema** - documents can have different structures
- **Scalable** - handles large amounts of data
- **Fast** - optimized for performance

Key difference: No tables, rows, or columns like SQL databases

SQL vs MongoDB

SQL Database	MongoDB
Database	Database
Table	Collection
Row	Document
Column	Field
Primary Key	_id (auto-generated)

MongoDB Structure

```
Database: labDB
  └── Collection: students
        ├── Document 1: { name: "Ali", age: 21, major: "CS" }
        ├── Document 2: { name: "Sara", age: 23, major: "SE" }
        └── Document 3: { name: "Ahmed", age: 20, major: "CS", gpa: 3.8 }
```

Note: Documents in the same collection can have different fields!

MongoDB Cloud (Atlas)

MongoDB Atlas is the **cloud-hosted** version of MongoDB

Benefits:

- Free tier available
- No local installation needed
- Accessible from anywhere
- Automatic backups
- Easy to scale

URL: https://cloud.mongodb.com/

Setting Up MongoDB Atlas

Step 1: Sign up and create a cluster

- Choose free plan
- Select AWS as provider
- Choose region and name the cluster

Step 2: Create database user

- Set username and password
- Save credentials securely

Step 3: Configure network access

- Add IP address: `0.0.0.0/0` (allows all IPs)

MongoDB Shell (mongosh)

mongosh is the command-line interface for MongoDB

Installation:

- Download from MongoDB website
- Or use MongoDB Atlas web shell

Purpose:

- Connect to MongoDB
- Execute commands
- Perform CRUD operations
- Manage databases

Connecting to MongoDB

From MongoDB Atlas:

1. Click "Connect" on your cluster
2. Choose "Shell" option
3. Copy connection string
4. Paste in terminal/command prompt

Connection string format:

```
mongodb+srv://username:password@cluster0.xxxxx.mongodb.net
```

Working with Databases

Create or switch to a database:

```
use labDB
```

Note: Database is created automatically when you first insert data

View current database:

```
db
```

List all databases:

```
show dbs
```

Working with Collections

Create a collection:

```
db.createCollection("students")
```

Note: Collection is also created automatically when you insert first document

List collections in current database:

```
show collections
```

Drop a collection:

```
db.students.drop()
```

MongoDB Documents

Documents are JSON-like objects stored in collections

```
{
  _id: ObjectId("..."),  // Auto-generated unique ID
  name: "Ali",
  age: 21,
  major: "CS"
}
```

Key points:

- `_id` is automatically created (unique identifier)
- Fields can be any data type
- Documents can have nested objects and arrays

CRUD Operations

CRUD = Create, Read, Update, Delete

These are the four basic operations for managing data:

- **Create**: Insert new documents
- **Read**: Find/query documents
- **Update**: Modify existing documents
- **Delete**: Remove documents

Create: Inserting Documents

Insert one document:

```
db.students.insertOne({
  name: "Ali",
  age: 21,
  major: "CS"
})
```

Insert multiple documents:

```
db.students.insertMany([
  { name: "Ali", age: 21, major: "CS" },
  { name: "Sara", age: 23, major: "SE" }
])
```

Returns: Object with `insertedId` or `insertedIds`

Read: Finding Documents

Find all documents:

```
db.students.find()
```

Find with filter:

```
db.students.find({ major: "CS" })
```

Find one document:

```
db.students.findOne({ name: "Ali" })
```

Read: Query Operators

Comparison operators:

```
db.students.find({ age: { $gt: 21 } })  // Greater than
db.students.find({ age: { $lt: 20 } })  // Less than
db.students.find({ age: { $gte: 21 } }) // Greater than or equal
db.students.find({ age: { $lte: 23 } }) // Less than or equal
```

Multiple conditions:

```
db.students.find({ major: "CS", age: { $gt: 20 } })
```

Update: Modifying Documents

Update one document:

```
db.students.updateOne(
  { name: "Ali" },            // Filter
  { $set: { age: 22 } }       // Update operation
)
```

Update multiple documents:

```
db.students.updateMany(
  {},                          // Filter (empty = all)
  { $inc: { age: 1 } }        // Increment age by 1
)
```

Common operators: `$set`, `$inc`, `$unset`, `$push`

Delete: Removing Documents

Delete one document:

```
db.students.deleteOne({ name: "Sara" })
```

Delete multiple documents:

```
db.students.deleteMany({ age: { $lt: 20 } })
```

Delete all documents (keep collection):

```
db.students.deleteMany({})
```

Delete collection:

```
db.students.drop()
```

Mongoose

What is Mongoose?

Mongoose is an **Object Data Modeling (ODM)** library for MongoDB and Node.js

Purpose:

- Provides schema validation
- Type casting
- Query building
- Middleware hooks
- Easy connection management

Think of it as: A bridge between Node.js and MongoDB

Why Use Mongoose?

Without Mongoose

- Manual validation
- No schema enforcement
- More verbose code
- Manual type conversion

With Mongoose:

- Schema-based validation
- Automatic type casting
- Cleaner, more readable code
- Built-in methods and helpers

Connecting to MongoDB with Mongoose

Connection string:

```
mongoose.connect(
  "mongodb+srv://username:password@cluster0.xxxxx.mongodb.net"
)
```

With options:

```
mongoose.connect(connectionString, {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
```

Connecting to MongoDB with Mongoose

Handle connection:

```
mongoose.connect(connectionString)
  .then(() => console.log("Connected to MongoDB"))
  .catch(err => console.log("Connection error:", err));
```

Mongoose Schemas

Schema defines the structure of documents

Example:

```
const studentSchema = new mongoose.Schema({
  name: String,
  age: Number,
  major: String
});
```

Schema types:

- `String`, `Number`, `Boolean`, `Date`
- `Array`, `Object`
- `mongoose.Schema.Types.ObjectId`

Mongoose Models

Model is a class compiled from a schema

Create a model:

```
const Student = mongoose.model("Student", studentSchema);
```

Model name: "Student" (singular, capitalized)

Collection name: "students" (plural, lowercase - auto-generated)

Use the model to perform CRUD operations

Schema with Options

Add validation and options:

```
const studentSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  age: {
    type: Number,
    min: 0,
    max: 150
  },
  major: {
    type: String,
    default: "Undeclared"
  }
});
```

Create: Inserting Documents

Using Mongoose:

```
async function createStudents() {
  await Student.insertMany([
    { name: "Ali", age: 21, major: "CS" },
    { name: "Sara", age: 23, major: "SE" }
  ]);
  console.log("Inserted successfully");
}

createStudents();
```

Create: Inserting Documents

Or create one:

```
const newStudent = new Student({
  name: "Ahmed",
  age: 20,
  major: "CS"
});
await newStudent.save();
```

Read: Finding Documents

Find all:

```
async function readStudents() {
  const all = await Student.find();
  console.log(all);
}
```

Read: Finding Documents

Find with filter:

```
const csStudents = await Student.find({ major: "CS" });
```

Find one:

```
const student = await Student.findOne({ name: "Ali" });
```

Find by ID:

```
const student = await Student.findById(id);
```

Read: Query Methods

Chain query methods:

```
const results = await Student
  .find({ major: "CS" })
  .where("age").gt(20)
  .limit(10)
  .sort({ age: -1 });
```

Common methods:

- `.limit(n)` - limit results
- `.sort({ field: 1 })` - sort (1 = asc, -1 = desc)
- `.select("name age")` - select specific fields
- `.where()` - add conditions

Update: Modifying Documents

Update one:

```
async function updateStudent() {
  await Student.updateOne(
    { name: "Ali" },
    { age: 22 }
  );
  console.log("Updated successfully");
}
```

Update: Modifying Documents

Update multiple:

```
await Student.updateMany(
  { major: "CS" },
  { $inc: { age: 1 } }
);
```

Find and update:

```
const student = await Student.findOneAndUpdate(
  { name: "Ali" },
  { age: 22 },
  { new: true }  // Return updated document
);
```

Delete: Removing Documents

Delete one:

```
async function deleteStudent() {
  await Student.deleteOne({ name: "Sara" });
  console.log("Deleted successfully");
}
```

Delete multiple:

```
await Student.deleteMany({ age: { $lt: 20 } });
```

Find and delete:

```
const student = await Student.findOneAndDelete({ name: "Ali" });
```

Mongoose vs mongosh

Feature	mongosh	Mongoose
Environment	Command line	Node.js
Schema	No validation	Schema validation
Type Safety	Manual	Automatic
Code Style	MongoDB syntax	JavaScript objects
Use Case	Testing, admin	Application code

Best Practices

Always use async/await:

```
async function main() {
  await mongoose.connect(connectionString);
  // ... operations
}
```

Best Practices

Handle errors:

```
try {
  await Student.create({...});
} catch (error) {
  console.error("Error:", error);
}
```

Close connection (if needed):

```
await mongoose.connection.close();
```

What's Next?

Practice:

- Set up MongoDB Atlas
- Try CRUD operations in mongosh
- Build a Node.js app with Mongoose

Wednesday lecture: RESTful APIs with Express and MongoDB

30m

Demo

7.1 mongoDB

MongoDB & Mongoose Hands-on

- MongoDB Atlas setup
- mongosh CRUD operations
- Mongoose connection and CRUD